From Abstracts to Rooms: Poster Session Assignment by Semantic Similarity and Optimisation

Enes Sejdini
BSc (Hons)

A dissertation submitted for the degree of
Master of Science in Data Science

Supervised by *Dr. Pascal Welke*

School of Computing and Communications
Lancaster University Leipzig

July, 2026

Declaration

I declare that the work presented in this dissertation is, to the best of my knowledge and belief, original and my own work. The material has not been submitted, either in whole or in part, for a degree at this, or any other university. Estimated word count is:

3276 (errors:2)

Name: **Enes Sejdini**
Date: **July, 2026**

TO FILL: your dissertation title

TO FILL: your full name, BSc (Hons).
School of Computing and Communications, Lancaster University

A dissertation submitted for the degree of *Master of Science* in Cyber Security.

July, 2026

Abstract

This is the beginning of the abstract. According to the Postgraduate Research Regulations 2022-23, it should not be any longer than 300 words. However, as your dissertation is to be submitted as part of a taught Master programme (which includes the MRes), there is no strict limit imposed on the abstract length. For general guidance on language, presentation, and ordering of your dissertation, refer to Appendix 2 (Pages 30-33) `https://bit.ly/2Q4H43I`**.**

Acknowledgements

Acknowledgements you may want to make. **(this is not required when submitting your dissertation before your viva and you can add a dedication in your final dissertation after your viva if you wish.)**

Contents

1 Introduction **1**
- 1.1 Motivation . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Relevance . . . 1
- 1.4 Proposed Approach . . . 1
- 1.5 Contributions . . . 1
- 1.6 Summary of Results . . . 1
- 1.7 Did It Work? . . . 1
- 1.8 Dissertation Structure . . . 1

2 Literature Review **2**
- 2.1 Conference Session and Poster Assignment . . . 2
- 2.2 Representing Scientific Text as Vectors . . . 3
- 2.3 Hierarchical Clustering of Documents . . . 3
- 2.4 Capacitated Grouping and Clique Partitioning . . . 4
- 2.5 Metaheuristics for Partitioning Problems . . . 4
- 2.6 Benchmarking Algorithms . . . 5
- 2.7 Summary and Positioning . . . 5

3 Pipeline Overview **6**
- 3.1 Design Goals . . . 6
- 3.2 Architecture . . . 6
- 3.3 Why This Decomposition . . . 7
- 3.4 Notation and Terminology . . . 8
- 3.5 Evaluation Strategy . . . 8

4 Data Collection **9**
- 4.1 Two Datasets and Their Roles . . . 9
- 4.2 The NeurIPS 2025 Dataset . . . 10
- 4.3 The Paraphrase Evaluation Set . . . 10
- 4.4 The Demonstration Dataset . . . 11

5 Embedding Model Selection **12**

6 Clustering **13**
6.1 Hierarchical Agglomerative Clustering 13
6.2 Over-Clustering 13
6.3 Results 13
6.4 Discussion: Cohesion, Not Silhouette 13

7 Room Assignment **14**

8 The Layout Tool **15**
8.1 Purpose and Requirements 15
8.2 Architecture 15
8.3 Validation 15
8.4 Automatic Configuration 15
8.5 Outputs 15
8.6 Reproducibility and Portability 15

9 Results, Discussion and Analysis **16**
9.1 The Final Layout 16
9.2 Per-Room Analysis 16
9.3 The Cost of Exact Fill 16
9.4 The Ranking of Methods Depends on the Budget 16
9.5 Objective and Similarity Do Not Always Agree 16
9.6 Interpreting the Magnitude 16
9.7 Against Expectations and Prior Work 16

10 Conclusion **17**
10.1 Summary 17
10.2 Limitations 17
10.3 Future Work 17

Appendix A Supplementary Material **18**
A.1 Room Capacities 18
A.2 Full Benchmark Tables 18
A.3 Cluster Examples 18
A.4 Code and Reproducibility 18

References **19**

List of Figures

3.1 The four stages of the pipeline. The input is any conference's papers as a csv file with one abstract per paper. NeurIPS 2025, retrieved from OpenReview, is the dataset used throughout this dissertation. The dashed arrows show the two non-linear parts: the evaluation of the finished layout against a random baseline, and the loop that re-cuts the cluster tree when a cluster does not fit the largest room. 7

4.1 Accepted NeurIPS 2025 papers per primary area, from 1,685 papers down to 54. 10
4.2 The 10,000 sampled arXiv papers per group, from 1,182 down to 1,001. . . . 11

List of Tables

4.1 The two datasets side by side. 9
4.2 Construction of the demonstration dataset. 11

Chapter 1

Introduction

1.1 Motivation

1.2 Problem Statement

1.3 Relevance

1.4 Proposed Approach

1.5 Contributions

1.6 Summary of Results

1.7 Did It Work?

1.8 Dissertation Structure

Chapter 2

Literature Review

2.1 Conference Session and Poster Assignment

The problem of organizing a scientific conference by computational means is not new. In an early study, Eglese and Rand (1987) surveyed the attendees of a 1985 conference to express which seminars they wished to attend, and then built a schedule that would allow the fullfillment of as many of these wants as possible. From that point on, the organizing of poster placements in a conference has become received a great deal of attention leading to the establishment of a large amount of operational research literature, where this problem has mainly been dealt as an integer linear problem. Research such as Stidsen, Pisinger, and Vigo (2018) describe a tool used to schedule the EURO-k conferences, events with more than two thousand presentations, by using optimization models to place related research buildings close to each other. Riquelme et al. (2022) focused on a "track-based" version of the problem—sorting choices into specific paths. Instead of finding an exact optimal solution, which is always computationally slow, they used a "simulated annealing heuristic," which is a smart shortcut algorithm. This shortcut found comparable solutions in a fraction of the time. Closest to our setting, Bulhões, Correia, and Subramanian (2022) define a general session scheduling problem whose objective is to maximize the similarity of papers put next to each other. They prove that this problem is NP-hard, and solve real conference instances with branch-and-cut and branch-cut-and-price methods. While this exact approach guarantees an optimal solution, it is computationally expensive, leaving room to develop faster methods that produce comparable results.

There is also a smaller line of research that finds topics by reading the actual paper texts, not by relying on pre-set labels or user preferences. Burke and Sabatta (2015) use Latent Dirichlet Allocation (LDA) directly to the submitted documents to learn the conference topics. They then schedule the parallel tracks so that simultaneous sessions cover the most different topics possible. By maximizing the distance between session topics, they ensure attendees are not forced to choose between two similar talks. In current practice, large

venues face the same task at much greater scale: For example, CVPR (CVPR, 2024) groups its accepted posters by similarity within each session. To decide which posters are similar, they use SPECTER embeddings and then reduce the size of those vectors down to a single line with one-dimensional t-SNE. Posters that land close together on that line are put in the same group . This is, to our knowledge, the closest real-world precedent for the problem we study; however, it is a projection heuristic rather than an explicit optimization, and the quality of the resulting layout is not measured.

Put together, the literature has largely pursued conflict avoidance and preference satisfaction. The topical coherence of a capacity-limited physical room, treated as an explicit optimization objective in itself, remains largely unexplored.

2.2 Representing Scientific Text as Vectors

Sentence embedding models map a text to a fixed-length vector such that semantically similar texts receive nearby vectors. Sentence-BERT introduced the now-standard approach of fine-tuning a pretrained transformer with siamese network structures, producing sentence vectors that can be compared directly with cosine similarity (Reimers and Gurevych, 2019). The model used later in this work, all-MiniLM-L6-v2, combines this training scheme with the MiniLM distillation method, which compresses a large teacher transformer into a six-layer student with 384-dimensional outputs (Wang et al., 2020). In contrast, a family of models exists specifically for scientific documents: SPECTER fine-tunes SciBERT on the citation graph, treating papers that cite one another as similar (Cohan et al., 2020); its successor SPECTER2 is trained on over six million citation triplets and attaches task-specific adapters (Singh et al., 2023); and Aspire models fine-grained similarity by matching individual sentences between papers (Mysore, Cohan, and Hope, 2022). The landscape is wider than the three models examined later: contrastive objectives such as SimCSE learn general-purpose sentence embeddings without any labelled pairs (**gao2021simcse**), and SciNCL replaces SPECTER's discrete citation relations with sampling over citation-graph embeddings (**ostendorff2022neighborhood**). These models were trained on different signals and therefore encode different notions of what makes two papers similar, a distinction whose practical consequences we examine in Chapter 5.

2.3 Hierarchical Clustering of Documents

Clustering methods vary not solely in quality but in their structural properties, and these properties determine how well a method fits a given task. Hierarchical agglomerative clustering starts from every point as its own group and repeatedly merges the closest pair, producing a tree that can afterwards be cut at any level to yield any number of clusters; Ward's method chooses each merge so that the increase of the within-group variance is

minimal, which favours compact groups (Ward, 1963). The k-means algorithm instead partitions the data into a number of groups that must be fixed in advance, and because it starts from a random initialization, its result can vary from run to run, which reduces reproducibility (MacQueen, 1967; Jain, 2010). Density-based methods such as HDBSCAN extract clusters of varying density from a hierarchy of density estimates (Campello, Moulavi, and Sander, 2013; McInnes, Healy, and Astels, 2017); a defining feature of this family is that points in low-density regions are labelled as noise and left outside every cluster. Finally, probabilistic topic models such as Latent Dirichlet Allocation(LDA) describe each document as a soft mixture of topics rather than assigning it to exactly one group (Blei, Ng, and Jordan, 2003).

In this work, clustering is judged by practical fit rather than benchmark accuracy: every poster must be placed (no unassigned outliers), the number of groups must be freely re-cut to match physical room capacities, and the outcome must be deterministic so that our layout heuristics remain stable. Chapter 6 derives the choice of hierarchical clustering directly from these three requirements.

2.4 Capacitated Grouping and Clique Partitioning

At its combinatorial core, our task is the clique partitioning problem: assign items to mutually exclusive groups so that the total similarity within groups is maximized. (Grötschel and Wakabayashi, 1989) introduced this formulation as a clustering model and proposed cutting plane methods for its solution. The problem was later shown to be NP-hard (Grötschel and Wakabayashi, 1990), and the geometric structure of its feasible solutions has been studied extensively (Oosten, Rutten, and Spieksma, 2001). Even with modern commercial solvers, an exact solution becomes impractical as instances grow, something that (Du et al., 2022) has also concluded through a direct comparison of models and solvers on clique partitioning benchmarks. Our problem in Chapter 7 adds two further restrictions to this core, namely room capacities and a minimum occupancy per room, and regardless of the added rules, it still doesn't remove its NP-hardness.

2.5 Metaheuristics for Partitioning Problems

For partitioning problems, a widely accepted set of metaheuristics exists. GRASP repeatedly builds a solution with a randomized greedy construction and then improves it with local search, keeping the best result over many restarts (Feo and Resende, 1995). Tabu search follows a single trajectory and uses a short-term memory of recently made moves to escape local optima (Glover and Laguna, 1997). Variable neighbourhood search alternates between descending to a local optimum and perturbing the solution with systematically growing shakes (Mladenović and Hansen, 1997). Memetic algorithms evolve a population of solutions

through recombination and mutation, with local search applied to the offspring (Moscato and Cotta, 2003). These methods have been applied to clique partitioning itself, for example through iterated tabu search (Palubeckis, Ostreika, and Tomkevičius, 2014), and hybrid approaches additionally exist that embed an exact solver inside a heuristic loop, a design we will go back to in Chapter 7. The metaheuristic family is of course wider than these five; simulated annealing, for instance, has been applied in the conference scheduling setting itself as well(Riquelme et al., 2022).

2.6 Benchmarking Algorithms

All of the methods above are randomized, so a single run of each says little about which one is better. The established standard procedure is therefore to compare distributions of results over multiple independent runs, and to test the differences with non-parametric statistics, since the outcomes of heuristics cannot be assumed to follow a normal distribution. Demšar (2006) recommends exactly this design: the Wilcoxon signed-rank test for comparing two methods on paired results, and the Friedman test as an omnibus test when several methods are compared at once. Running many tests together creates a problem of its own: each test has a small chance of a false alarm, so across ten pairwise comparisons at least one "significant" result is quite likely to be fake. The procedure of Holm (1979) fixes this by making each individual test stricter, keeping the whole table at the usual five percent error level, and García and Herrera (2008) show how to apply it when every method is compared against every other, which is our case. Chapter 7 follows exactly this protocol; it is also why every method there receives the same time budget and the same random seeds, since the tests compare methods seed by seed, and that pairing is only fair if both methods faced identical conditions on each seed.

2.7 Summary and Positioning

Together, the literature is divided into three paths.First, the operational research norm schedules conferences well, but its objectives are preventing overlapping sessions and satisfying preferences, not the topical coherence of a room. Second, a smaller line of research reads the papers themselves to find topics but stops at abstract groupings without rooms or seat capacities. Third, in practice, large venues already assign posters with embedding-based heuristics, without measuring the quality of the result. All building blocks our pipeline needs exists on its own: sentence embeddings, hierarchical clustering, the clique partitioning problem, metaheuristics, and a statistical protocol for comparing them fairly. What no prior work does, is combine them, a reproducible pipeline that takes a conference's raw submissions and returns a topically coherent, physically feasible assignment of papers to capacity-limited rooms. That arrangement is what this dissertation provides.

Chapter 3

Pipeline Overview

3.1 Design Goals

The pipeline was built to meet five requirements. First, the layout it produces must be topically coherent: papers found in the same room should talk about similar concepts, since this is the entire purpose of the system itself. Second, the layout must be physically feasible, meaning that no room is allowed to hold more papers than it has seats and no room is allowed to be left empty. Third, the pipeline must maintain peak performance at the scale of a real conference, which today means thousands of papers. Fourth, it must be general: the same pipeline, standardized, should accept any conference's papers and any set of room sizes, ensuring reliability at any scale of a scientific conference. Finally, it must be reproducible: every random choice the pipeline makes is controlled by a fixed seed, so repeated runs follow exactly the same search. However, in the heuristic faze only the search is stopped by wall-clock time rather than by a step count, the resulting layout is the same in practice but not guaranteed to be identical, a point Chapter 8 returns to.

3.2 Architecture

The pipeline is made up of four stages, shown in Figure 3.1. First stage obtains the accepted papers of a conference, in our case the abstracts and corresponding metadata of the accepted NeurIPS 2025 submissions. Embedding starts the second stage by converting each abstract into a fixed-length numerical vector, so that the similarity of two papers can be computed as the similarity of their vectors. Clustering groups these vectors into many small topic clusters, by calculating the distance of the vectors with eachother and grouping the closest ones together. Room assignment then packs whole clusters into the available rooms, maximizing the similarity of the clusters that end up together, and an evaluation step measures the quality of the finished layout. Each stage hands its output to the next: stage one delivers the

abstracts, stage two turns them into unit-length vectors, stage three groups the vectors into clusters with centroids, and stage four turns the clusters into a paper-to-room layout again based on similarity of papers between clusters.

The four stages follow the standard data science pipeline of collection, representation, modelling and validation. As that framework itself emphasizes, the process is iterative rather than linear: in our pipeline, if clustering produces a cluster larger than the largest room, no feasible layout can exist, and the flow loops back to re-cut the same cluster tree more finely until every cluster fits. The pipeline is demonstrated throughout this dissertation on a single real instance, the 5,286 accepted papers of NeurIPS 2025 assigned to twelve rooms, and the properties of this dataset are described in Chapter 4.

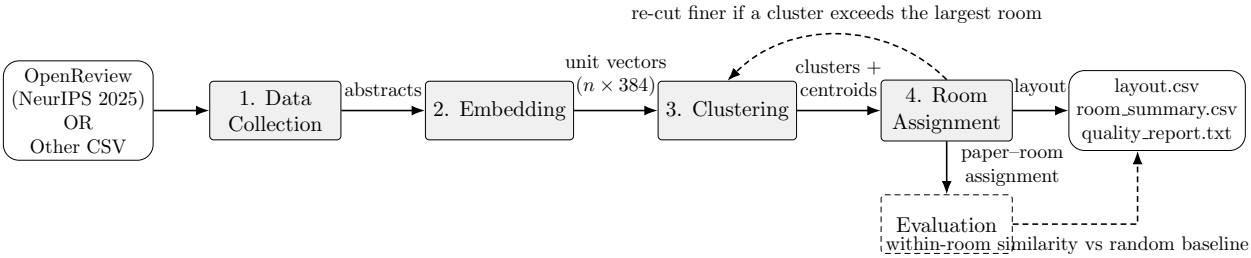

Figure 3.1: The four stages of the pipeline. The input is any conference's papers as a csv file with one abstract per paper. NeurIPS 2025, retrieved from OpenReview, is the dataset used throughout this dissertation. The dashed arrows show the two non-linear parts: the evaluation of the finished layout against a random baseline, and the loop that re-cuts the cluster tree when a cluster does not fit the largest room.

3.3 Why This Decomposition

The central design decision of the pipeline is that papers are not assigned to rooms individually. Instead, they are first grouped into clusters, and the optimization then places the clusters into the given rooms. Three reasons for doing this. The first is scale: assigning 5,286 individual papers to twelve rooms encompasses a search space far greater than what any exact or heuristic method can explore meaningfully in a short time, while a few hundred clusters can simplify the task significantly. The second is signal: a single paper provides little topical information on its own, whereas a cluster of related papers can end up having a stable, recognizable subject, making it easier to optimize. The third is past experiences: nobody walks a poster session with a list of single papers in mind. People think in topics, see the robotics posters, then the medical ones, so the natural focus of a session area the paper subject areas, and the produced clusters end up being building blocks of those areas. Assigning papers one by one could sometimes scatter a topic across several rooms, assigning

whole clusters increases the likelyhood of every topic being in the right place.

This decomposition has an honest cost, and we state it here because it is a property of the shape rather than of any individual method: clusters are unchanging. Once formed, a cluster is decided, so the grouping might interfeer with the flexibility of splitting a topic accross two different rooms. This lack of flexibility is what later makes filling every room to its exact seat count difficult, and its consequences are examined in Chapter 9. Further details on which clustering method forms the clusters, and which optimization method packs them, are also treated in Chapter 6 and Chapter 7 respectively.

3.4 Notation and Terminology

The following notation is used throughout. The conference has n papers and R rooms with seat capacities $c_1, \ldots, c_R$. Clustering produces C clusters with sizes $s_1, \ldots, s_C$, each represented by a centroid, and W denotes the $C \times C$ table of pairwise centroid similarities. A layout is an assignment that maps every cluster to exactly one room. Two terms are kept strictly apart: the *objective* is the cluster-level score that the optimization maximizes, while the *within-room similarity* is the paper-level measure by which the finished layout is assessed. These are all related but not identical measurements, and further details on the importance of each is analysed in Chapter 9. However, the formal optimization model built from this notation is stated at the start of Chapter 7.

3.5 Evaluation Strategy

The pipeline is evaluated using three distinct measures. The first is the optimization objective, which serves as the internal mathematical score that the assignment methods in Chapter 7 compete to maximize. The second is the mean within-room similarity of the actual papers. Because this is calculated independently of the optimizer, it acts as an unbiased reality check of the grouping quality. The third is a baseline control, created by randomly shuffling the exact same papers into the same room sizes. Because all papers at a single conference naturally share a broad topic and vocabulary, any random grouping will inherently yield a baseline level of similarity. Consequently, a raw similarity score is meaningless on its own; it only becomes interpretable when compared directly against this random baseline to prove the algorithm is genuinely finding deeper connections.

Chapter 4

Data Collection

4.1 Two Datasets and Their Roles

This work uses two datasets with strictly separated roles; Table 4.1 compares them. Every experiment and every design decision, from the model comparison of Chapter 5 to the final layout of Chapter 8, is made on the NeurIPS 2025 corpus. The arXiv sample is used exactly once, in Chapter 8, to run the finished tool on a corpus it was not developed on; nothing touches it before that run. A third, small set is derived from the NeurIPS data: the paraphrase set of Section 4.3.

Table 4.1: The two datasets side by side.

	NeurIPS 2025	**arXiv sample**
Source	OpenReview API	arXiv metadata snapshot
Papers	5,286	10,000
Fields	5	4
Missing values	none	none
Fields the tool reads	Title, Abstract	title, abstract
Reference labels	Primary Area	group, subcategory
Abstract mean	181.6 words	138.4 words
Abstract range	37–516 words	5–344 words
Categories	13 primary areas	9 arXiv groups
Balance	heavily unbalanced	close to balanced
Snapshot	July 2026, frozen	frozen sample, fixed seed
Role	all development	demonstration only

4.2 The NeurIPS 2025 Dataset

The papers come from OpenReview, the platform on which NeurIPS runs its review process. A short Python script, using the official OpenReview client library, requests all submissions carrying the venue identifier NeurIPS.cc/2025/Conference and writes them to one CSV file, one row per paper. The accepted NeurIPS 2025 submissions were downloaded in July 2026, and the dataset reflects the state of the platform at that time. Only the abstract feeds the pipeline; the title labels the outputs, and the rest is kept as reference only.

Figure 4.1 shows the categories the authors chose for their papers. Deep learning alone holds 1,685 papers, almost a third of the conference, while the three smallest areas hold fewer than sixty each. Because most papers here are already similar, our implementation can show false success thus leading to the need of a second testing dataset.

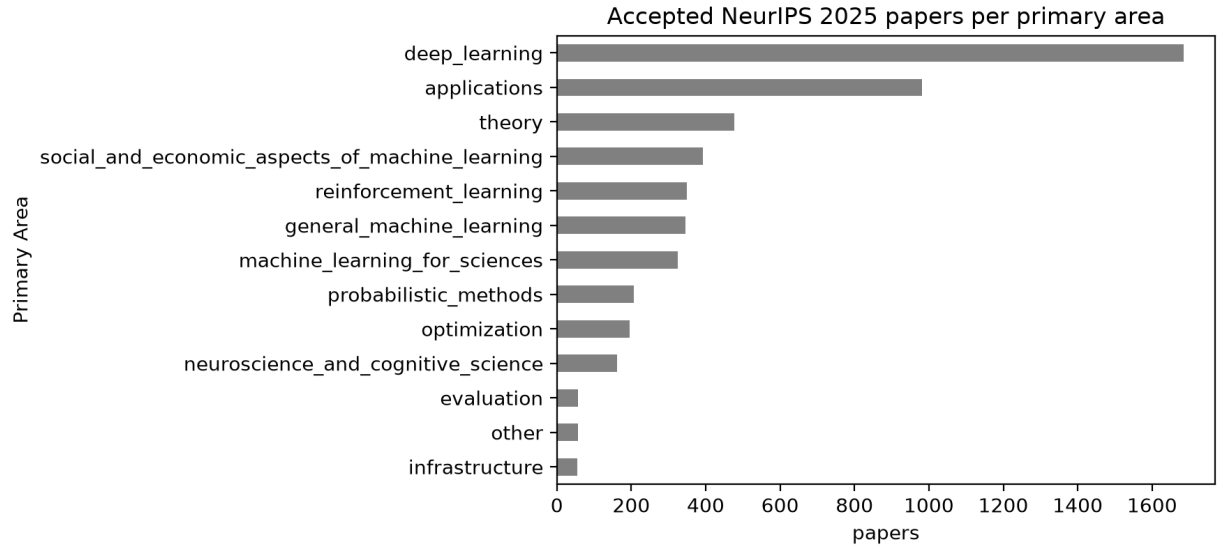

Figure 4.1: Accepted NeurIPS 2025 papers per primary area, from 1,685 papers down to 54.

4.3 The Paraphrase Evaluation Set

Chapter 5 needs a labelled test set for its model comparison. We built one from the NeurIPS data: 250 random abstracts, each rewritten into a single sentence by the Gemma E2B model, run locally through Ollama. A good model should recognise the original paper from that sentence. The words change, the meaning does not. Rerunning the model gives different sentences each time, so the set was written once and kept unchanged.

4.4 The Demonstration Dataset

The NeurIPS corpus is topically narrow, almost a third of it is deep learning. High within-room similarity could come partly from the papers resembling each other, not from the implementation being good. The finished application is therefore also run once, deliberately, through different dataset, built as Table 4.2 describes. Figure 4.2 shows the outcome: nine fields, close to balanced, the opposite of Figure 4.1.

Table 4.2: Construction of the demonstration dataset.

Source	arXiv metadata snapshot, ∼3 million entries
Selected	the nine top-level arXiv groups
Category used	primary, the first one the authors listed
Excluded	legacy categories from the 1990s
Cap per group	11,000, the size of the smallest group
Within a group	cap split equally among subcategories; undersized ones contribute all they have
Parent set	[TO FILL] papers
Sample	10,000, uniform random, fixed seed
Provided	sampled file ships with the code

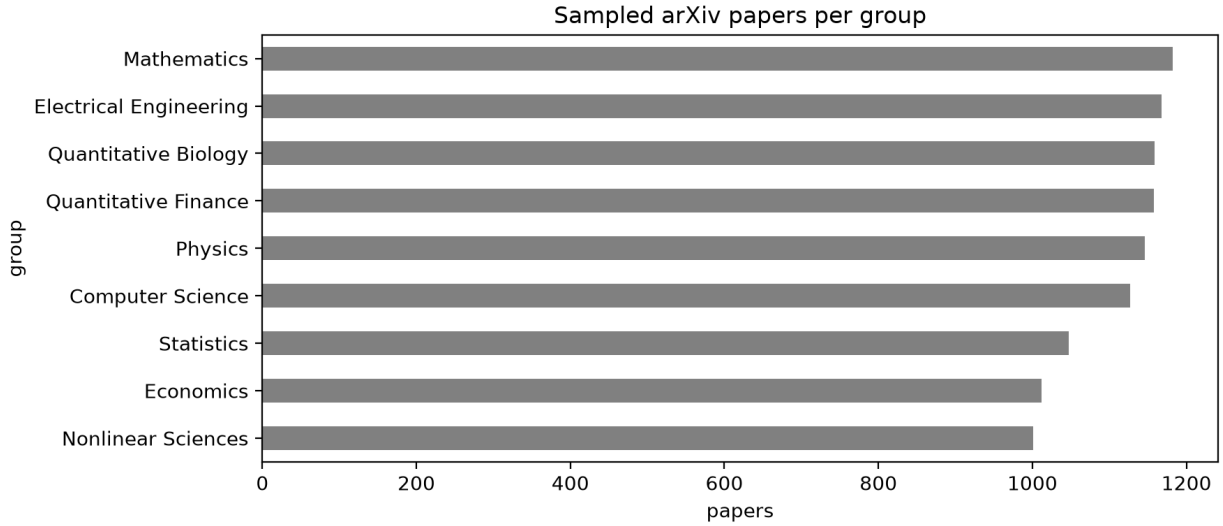

Figure 4.2: The 10,000 sampled arXiv papers per group, from 1,182 down to 1,001.

Chapter 5

Embedding Model Selection

Chapter 6

Clustering

6.1 Hierarchical Agglomerative Clustering

6.2 Over-Clustering

6.3 Results

6.4 Discussion: Cohesion, Not Silhouette

Chapter 7

Room Assignment

Chapter 8

The Layout Tool

8.1 Purpose and Requirements

8.2 Architecture

8.3 Validation

8.4 Automatic Configuration

8.5 Outputs

8.6 Reproducibility and Portability

Chapter 9

Results, Discussion and Analysis

9.1 The Final Layout

9.2 Per-Room Analysis

9.3 The Cost of Exact Fill

9.4 The Ranking of Methods Depends on the Budget

9.5 Objective and Similarity Do Not Always Agree

9.6 Interpreting the Magnitude

9.7 Against Expectations and Prior Work

Chapter 10

Conclusion

10.1 Summary

10.2 Limitations

10.3 Future Work

Appendix A

Supplementary Material

A.1 Room Capacities

A.2 Full Benchmark Tables

A.3 Cluster Examples

A.4 Code and Reproducibility

References

Blei, David M., Andrew Y. Ng, and Michael I. Jordan (2003). "Latent Dirichlet Allocation". In: *Journal of Machine Learning Research* 3, pp. 993–1022.

Bulhões, Teobaldo, Rubens Correia, and Anand Subramanian (2022). "Conference scheduling: A clustering-based approach". In: *European Journal of Operational Research* 297.1, pp. 15–26. DOI: 10.1016/j.ejor.2021.04.042.

Burke, Michael and Deon Sabatta (2015). "Topic models for conference session assignment: organising PRASA 2014". In: *Proceedings of the 2015 Pattern Recognition Association of South Africa and Robotics and Mechatronics International Conference (PRASA-RobMech)*.

Campello, Ricardo J. G. B., Davoud Moulavi, and Jörg Sander (2013). "Density-Based Clustering Based on Hierarchical Density Estimates". In: *Advances in Knowledge Discovery and Data Mining (PAKDD)*, pp. 160–172. DOI: 10.1007/978-3-642-37456-2_14.

Cohan, Arman et al. (2020). "SPECTER: Document-level Representation Learning using Citation-informed Transformers". In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*.

CVPR (2024). *CVPR 2024 Accepted Papers*. URL: https://cvpr.thecvf.com/Conferences/2024/AcceptedPapers (visited on 07/16/2026).

Demšar, Janez (2006). "Statistical Comparisons of Classifiers over Multiple Data Sets". In: *Journal of Machine Learning Research* 7, pp. 1–30.

Du, Yu et al. (2022). "Solving clique partitioning problems: A comparison of models and commercial solvers". In: *International Journal of Information Technology & Decision Making* 21, pp. 59–81. DOI: 10.1142/S0219622021500504.

Eglese, R. W. and G. K. Rand (1987). "Conference Seminar Timetabling". In: *Journal of the Operational Research Society* 38.7, pp. 591–598.

Feo, Thomas A. and Mauricio G. C. Resende (1995). "Greedy Randomized Adaptive Search Procedures". In: *Journal of Global Optimization* 6.2, pp. 109–133. DOI: 10.1007/BF01096763.

García, Salvador and Francisco Herrera (2008). "An Extension on "Statistical Comparisons of Classifiers over Multiple Data Sets" for all Pairwise Comparisons". In: *Journal of Machine Learning Research* 9, pp. 2677–2694.

Glover, Fred and Manuel Laguna (1997). *Tabu Search*. Springer. DOI: 10.1007/978-1-4615-6089-0.

Grötschel, Martin and Yoshiko Wakabayashi (1989). "A cutting plane algorithm for a clustering problem". In: *Mathematical Programming* 45, pp. 59–96. DOI: 10.1007/BF01589097.

— (1990). "Facets of the clique partitioning polytope". In: *Mathematical Programming* 47, pp. 367–387.

Holm, Sture (1979). "A Simple Sequentially Rejective Multiple Test Procedure". In: *Scandinavian Journal of Statistics* 6.2, pp. 65–70.

Jain, Anil K. (2010). "Data clustering: 50 years beyond K-means". In: *Pattern Recognition Letters* 31.8, pp. 651–666.

MacQueen, James (1967). "Some methods for classification and analysis of multivariate observations". In: *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*. Vol. 1, pp. 281–297.

McInnes, Leland, John Healy, and Steve Astels (2017). "hdbscan: Hierarchical density based clustering". In: *Journal of Open Source Software* 2.11, p. 205. DOI: 10.21105/joss.00205.

Mladenović, Nenad and Pierre Hansen (1997). "Variable neighborhood search". In: *Computers & Operations Research* 24.11, pp. 1097–1100.

Moscato, Pablo and Carlos Cotta (2003). "A Gentle Introduction to Memetic Algorithms". In: *Handbook of Metaheuristics*. Ed. by Fred Glover and Gary A. Kochenberger. Kluwer.

Mysore, Sheshera, Arman Cohan, and Tom Hope (2022). "Multi-Vector Models with Textual Guidance for Fine-Grained Scientific Document Similarity". In: *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*.

Oosten, Maarten, Jeroen H. G. C. Rutten, and Frits C. R. Spieksma (2001). "The clique partitioning problem: Facets and patching facets". In: *Networks* 38.4, pp. 209–226.

Palubeckis, Gintaras, Armantas Ostreika, and Arūnas Tomkevičius (2014). "An Iterated Tabu Search Approach for the Clique Partitioning Problem". In: *The Scientific World Journal* 2014, p. 353101.

Reimers, Nils and Iryna Gurevych (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 3982–3992.

Riquelme, Fabián et al. (2022). "A Track-Based Conference Scheduling Problem". In: *Mathematics* 10.21, p. 3976.

Singh, Amanpreet et al. (2023). "SciRepEval: A Multi-Format Benchmark for Scientific Document Representations". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*.

Stidsen, Thomas, David Pisinger, and Daniele Vigo (2018). "Scheduling EURO-k conferences". In: *European Journal of Operational Research* 270.3, pp. 1138–1147.

Wang, Wenhui et al. (2020). "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers". In: *Advances in Neural Information Processing Systems*. Vol. 33.

Ward, Joe H. (1963). "Hierarchical Grouping to Optimize an Objective Function". In: *Journal of the American Statistical Association* 58.301, pp. 236–244. DOI: 10.1080/01621459.1963.10500845.